Assignment No. 7

Name: Shubham Shivaji Phad
Class & Division: LY AIA 03 Batch B
Roll-No:48
Enrollment Number: ADT23SOCB1106

Title

Design a syntactic analysis system that performs constituency parsing and dependency parsing to discover grammatical relationships among words in complex sentences.

Aim

To implement a syntactic analysis system using constituency parsing and dependency parsing to identify the grammatical structure and relationships among words in complex sentences.

Objectives

- Understand syntactic analysis in Natural Language Processing.
- Perform constituency parsing to represent sentence structure as phrases.
- Perform dependency parsing to identify grammatical relationships among words.
- Analyze a complex sentence using NLTK and SpaCy.

Theory

Syntactic analysis studies the grammatical structure of a sentence. **Constituency parsing** represents a sentence as nested constituents such as noun phrases and verb phrases. **Dependency parsing** represents grammatical relationships between individual words using a head-dependent structure.

NLTK provides tools for parsing and corpus handling, while SpaCy provides dependency parsing as part of its NLP pipeline. These techniques help in understanding sentence structure and grammatical relationships.

Software/Libraries Required

- Python 3.x
- Google Colab / Jupyter Notebook

- NLTK
- SpaCy
- SpaCy English model: `en_core_web_sm`

Installation Commands

```
!pip install nltk spacy -q
!python -m spacy download en_core_web_sm -q
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 88.3 MB/s eta 0:00:00
✔ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting
the
'Restart kernel' or 'Restart runtime' option.
```

Algorithm/Steps

1. Install NLTK and SpaCy with the English SpaCy model.
2. Define a complex sentence for analysis.
3. Create a context-free grammar and use NLTK ChartParser for constituency parsing.
4. Display the resulting constituency tree.
5. Load the SpaCy English model and process the same sentence.
6. Extract each token, its part of speech, syntactic dependency, and head word.
7. Display the dependency relationships.

Program with Output

1. Constituency Parsing using NLTK

```
import nltk
from nltk import CFG, ChartParser

sentence = "The intelligent student solved the difficult problem."
tokens = sentence.lower().replace('.', '').split()

grammar = CFG.fromstring('''
S -> NP VP
NP -> Det Adj N | Det N
VP -> V NP
Det -> 'the'
Adj -> 'intelligent' | 'difficult'
N -> 'student' | 'problem'
V -> 'solved'
''')

parser = ChartParser(grammar)
trees = list(parser.parse(tokens))

print("Constituency Parse Tree:")
```

```
for tree in trees:
    print(tree.pformat())
    tree.pretty_print()

Constituency Parse Tree:
(S
  (NP (Det the) (Adj intelligent) (N student))
  (VP (V solved) (NP (Det the) (Adj difficult) (N problem))))
                          S
          ________________|_________
         |                          VP
         |                  ________|______
         NP                |               NP
  _______|_________         |      _________|________
Det     Adj        N        V    Det       Adj       N
 |       |         |        |     |         |        |
the intelligent student solved the     difficult problem
```

Constituency Output

The sentence is represented using nested constituents such as `S`, `NP`, and `VP`.

2. Dependency Parsing using SpaCy

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp(sentence)

print("Dependency Parse:")
print(f"{'Word':<15}{'POS':<10}{'Dependency':<15}{'Head'}")
print("-" * 55)
for token in doc:
    print(f"{token.text:<15}{token.pos_:<10}{token.dep_:<15}
         {token.head.text}")

Dependency Parse:
Word           POS       Dependency     Head
-------------------------------------------------------
The            DET       det            student
intelligent    ADJ       amod           student
student        NOUN      nsubj          solved
solved         VERB      ROOT           solved
the            DET       det            problem
difficult      ADJ       amod           problem
problem        NOUN      dobj           solved
.              PUNCT     punct          solved
```

Dependency Output

The output displays each word with its POS tag, dependency relation, and syntactic head. For example, the verb `solved` acts as the main predicate, while `student` is connected to it as the subject and `problem` as the object.

The exact dependency labels can vary slightly with the SpaCy model version.

Conclusion

The syntactic analysis system was successfully implemented using constituency parsing and dependency parsing. Constituency parsing represented the sentence as nested grammatical phrases, while dependency parsing identified relationships between words and their syntactic heads. These techniques are useful for understanding grammatical structure in NLP applications.